Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr. 8

Disciplina: Tehnici de programare

Tema: ”Colecții: liste,tupluri,mulțimi,dicționare
Privesc pe Linux: enumerare spre root ”

A efectuat : st. gr.SI –263 Boicu Ecaterina

A verificat : asistent universitar Reițman Patricia

Chișinău – 2026

SARCINA LUCRĂRII

Scopul lucrării este exersarea celor patru colecții de bază din Python (lista,tuplul,mulțimea și dicționarul),împreuna cu comprehenshion-urile , pe o problemă de securitate informațională: enumerarea posibilităților de extindere a privilegiilor spre root pe un sistem Linux compromis.

Lucrarea are două părți:

Lucrarea A: unealta enum.py citește trei capturi (fișiere SUID , reguli sudo , sarcini cron), așază constatările în colecții potrivite și produce un rezumat pe categorii și un raport JSON.

Lucrarea B: unealta diferenta.py compară fișierile SUID de pe mașină analizată cu o instalare curată , de referință , folosind diferența de mulțimi și scoate fișierile noi și periculoase. Un test cu pytest verifică operația-cheie.

Datele sunt capturi de la o mașină de laborator izolată.Unealta citește fișiere , nu ruleaza comenzi pe un sistem real.

Fișiere de intrare:

Folderul start/ conține patru capturi , plasate în arsenal/privesc/start:

Fișier	Conținut	Comanda de Obținere pe o mașină reală
suid.txt	Fișiere cu bitul SUID (14 rânduri, cu o dublură)	find / -perm -4000 -type f 2>/dev/null
sudo.txt	Drepturile sudo ale conturilor (3 rânduri)	sudo -1
crontab.txt	Sarcini programate de sistem (3 sarcini și un comentariu)	cat /etc/crontab
baza_suid.txt	Aceleași SUID pe o instalare curată (9 rânduri)	referință pentru Lucrarea B

Fig .1 - Fișiere de intrare

Strucutra proiectului

Lucrarea este în depozitul de curs, în subfolderul arsenal/privesc/ , fără a modifica lucrările anterioare:

```
Tehnici de Programare/
└── arsenal/
    ├── __init__.py
    └── privesc/
        ├── __init__.py
        ├── start/            (suid.txt,
sudo.txt, crontab.txt, baza_suid.txt)
        ├── enum.py           (Lucrarea A)
        ├── enum.json         (raportul generat)
        ├── diferenta.py      (Lucrarea B)
        └── test_diferenta.py (Lucrarea B, test)
```

Fig.1.1 - Structura schematică a proiectului

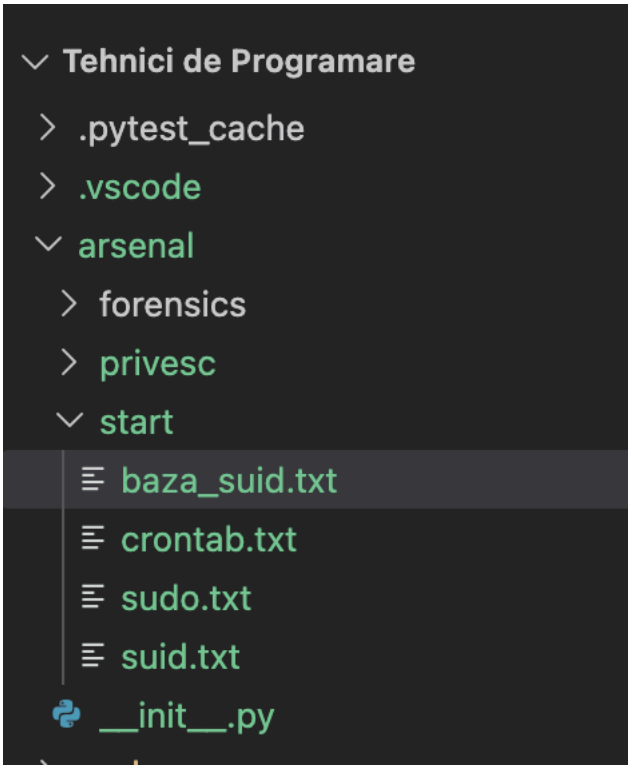


Fig.1.2 - Folderul Tehnici de Programare din VS Code

Fișierele __init__.py fac din foldere pachete Python , astfel încât diferenta.py poate importa arsenal.privesc.enum .

Colecții folosite și motivul alegerii

Colecții	Unde apare în lucrare?	De ce a fost aleasă?
Listă	Linii citite din fișier;listele de constatări	Păstrează ordinea și dublurile , iar raportul are nevoie de ordine
Tuplul	user, drept = ln.split(“ : ”,1)	O linie din sudo.txt se despachetează în două valori fixe
Mulțime	Numele unice , GTFO , userii unici , diferența acum - baza	elimină dublurile , căutarea cu in este rapidă , iar operatorii - și | înlocuiesc buclele
Dicționar	Fiecare constatare; raportul întreg	grupează date cu chei numite și dă un JSON natural

Fig.1.3- Colecții folosite

De asemenea , au fost folositecomprehension-urile : list comprehension cu if (filtrarea) , set comprehension (nume unice) și dict comprehension (fișa nume —> risc).

Mediul de lucru

Lucrarea a fost realizată în VS Code pe macOS cu Python 3.9 și un mediu virtual

```
python3 -m venv .venv
source .venv/bin/activate
pip install pytest
```

Fig.1.4- Exemplu schematic de instalare a pytest și activarea .venv/

Folderul .venv/ a fost adăugat în .gitignore , împreună cu __pycache__/, .pytest_cache/ și .DS_Store. Toate rulările se fac din rădăcina depozitului , cu opțiunea -m :

python -m arsenal.privesc.enum arsenal/privesc/start

ETAPELE LUCRĂRII

LUCRAREA A : enum.py

A1. Citirea liniilor într-o listă

Funcția citeste_linii întoarce liniile curățate: fără spații în capete , fără rânduri goale și fără comentarii (#). Rezultatul pe suid.txt este 14 linii , inclusiv dublura /usr/bin/passwd/ pentru că lista păstrează tot .

```
(.venv) ekaterinalungul@MacBook-Air-Ekaterina Tehnici de Programare % pyth
on -m arsenal.privesc.enum
linii SUID: 14
(.venv) ekaterinalungul@MacBook-Air-Ekaterina Tehnici de Programare %
```

Fig.2.1 - Rezultatul itemului A1

A2. Nume unice cu mulțimea

Funcția nume întoarce ce urmează după ultimul / (cale.rsplit("/",1)[-1]) . Un set comprehension {nume(c) for c in cai } aruncă dublura și dă 13 nume distincte .



Fig.2.2- Rezultatul itemului A2

A3. Constatările SUID : listă de dicționare

Pentru fiecare cale unică , în ordine sortată , se construiește un dicționar cu cheile cale , nume, și risc . Riscul este "mare" dacă numele se află în mulțimea GTFO (binare cunoscute ca scăpări spre root , în stilul GTFOBins). Rezultat : 13 SUID , dintre care 4 cu risc mare: cp , find , nmap, vim.basic .

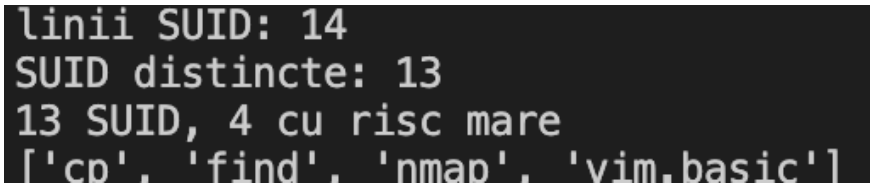


Fig.2.3- Rezultatul itemului A3

A4.sudo,cron și raportul grupat

sudo: linia ana: (ALL) NOPASSWD: /usr/bin/vim.basic se depsarte prin : și se despachetează într-un tuplu (user, drept).Riscul este “mare ” dacă conține NOPASSWD.

cron: linia se desparte după spații și tab-uri. Câmpurile 0-4 sunt programul , câmpul 5 este userul , iar restul este comanda.

enumereaza: adună cele trei liste într-un dicționar cu câte o cheie pe categorie (suid,sudo,cron).

A5. main() , rezumatul și JSON

Programul primește folderul capturilor și comutatorul - - json. Fără - - json afișează un rezumat pe categorii (cu .items()),mulțimea utilizatorilor implicați (userii din sudo reuniți cu cei din cron) și salvează raportul în enum.json. Pentru că sarcinile cron nu au cheia risc, numărarea folosește c.get(“risc”), care întoarce None în loc să de-a eroare.

```
suid: 13 gasite, 4 cu risc mare
sudo: 3 gasite, 2 cu risc mare
cron: 3 gasite, 0 cu risc mare
utilizatori implicati: ana, root, www-data
Raport salvat in enum.json
```

Fig 2.4 - Rezultatul itemului A5

Extras din enum.json (JSON valid , dicționar de liste de Dicționare):

```
    {
      "cale": "/bin/cp",
      "nume": "cp",
      "risc": "mare"
    },
    {
      "cale": "/bin/mount",
      "nume": "mount",
      "risc": "normal"
    },
    {
      "cale": "/bin/su",
      "nume": "su",
      "risc": "normal"
    },
    {
      "cale": "/bin/umount",
      "nume": "umount",
      "risc": "normal"
    },
    {
      "cale": "/usr/bin/chfn",
      "nume": "chfn",
      "risc": "normal"
    },
    {
      "cale": "/usr/bin/chsh",
      "nume": "chsh",
      "risc": "normal"
    },
    {
      "cale": "/usr/bin/find",
      "nume": "find",
      "risc": "mare"
    },
    {
      "cale": "/usr/bin/gpasswd",
      "nume": "gpasswd",
      "risc": "normal"
    },
```


Fig.2.5.1



```
      "cale": "/usr/bin/newgrp",
      "nume": "newgrp",
      "risc": "normal"
    },
    {
      "cale": "/usr/bin/nmap",
      "nume": "nmap",
      "risc": "mare"
    },
    {
      "cale": "/usr/bin/passwd",
      "nume": "passwd",
      "risc": "normal"
    },
    {
      "cale": "/usr/bin/sudo",
      "nume": "sudo",
      "risc": "normal"
    },
    {
      "cale": "/usr/bin/vim.basic",
      "nume": "vim.basic",
      "risc": "mare"
    }
  ],
  "sudo": [
    {
      "user": "ana",
      "drept": "(ALL) NOPASSWD: /usr/bin/vim.
      "risc": "mare"
    },
    {
      "user": "ana",
      "drept": "(ALL) /usr/bin/apt",
      "risc": "normal"
    },
    {
      "user": "www-data",
      "drept": "(root) NOPASSWD: /usr/bin/fin
      "risc": "mare"
```


Fig.2.5.2



Fig.2.5.3 - Extragerea din enum.json

Singura abatere față de schelet: raportul se salvează lângă enum.py (arsenal/privesc/enum.json) , nu în directorul curent , pentru ca fișierul să ajungă în locul cerut de lista de livrabile , indiferent de unde se rulează programul .

LUCRAREA B: diferent.py

B1.Două mulțimi

Funcția cai întoarce mulțimea căilor dintr-un fișier.Rezultat : baza are 9 elemente , acum are 13 (dublura din suid.txt dispare).

B2.Diferența

Expresia noi = acum - baza scoate fișierile SUID apărute față de sistemul curat , cu un singur operator , fără buclă dublă. Rezultat : 4 fișiere noi .

B3. Dicționar de risc și filtrul final

Un dict comprehension construiește fișa nume -> risc , iar un list comprehension cu if filtrează fișierile periculoase. Funcțiile nume și mulțimea GTFO sunt refolosite din enum.py printr-un import.

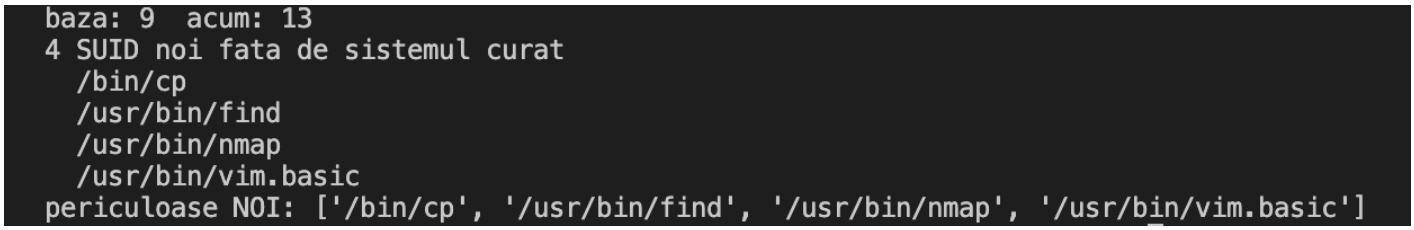


Fig.3.1 - Rezultatele B1-B3

B4.Testul

test_diferenta.py

```
def test_doar_ce_e_nou():
    acum = {"a", "b", "c"}
    baza = {"a", "b"}
    assert acum - baza == {"c"}
```

Fig.3.2.1- codul test_diferenta.py



Fig.3.2.2 - rularea prin: pytest -q arsenal/privesc/test_diferenta.py , cu rezultatul 1 passed.

Testul folosește date scrise de mână și nu are nevoie de rețea.

VERIFICAREA REZULTATELOR :

Verificare	Așteptat (fișa)	Obținut
Linii SUID citite	14	14
Nume SUID distincte	13	13
SUID cu risc mare	4(cp,find,nmap, vim.basic)	4(cp,find,nmap, vim.basic)
sudo:găsite /risc mare	3/2	3/2
cron:găsite/risc mare	3/0	3/0
Utilizatorii implicați	ana,root,www-data	ana,root,www-data
baza/acum	9/13	9/13
SUID noi față de baza curată	4	4
Test pytest	1 passed	1 passed

Fig.3.3 - Rezultatele finale verificate cu cele propuse în lucrare

CONCLUZII

În cadrul acestui laborator am lucrat cu cele patru colecții de bază din Python (list, tuplul, mulțimea și dicționarul),aplicate pe o problemă reală de securitate informațională : enumerarea posibilităților de extindere a privilegiilor pe un sistem Linux. Am utilizat două unelte, enum.py și diferenta.py , și am verificat că rezultatele coincid cu cele din fișa lucrării . Cele 13 fișiere SUID , dintre care 4 cu risc mare , drepturile sudo și sarcinile cron au fost grupate într-un raport JSON , iart testul pytest a trecut.

Am văzut că alegerea corectă a colecției simplifică codul : mulțimea elimină dublurile și face diferența dintre două stări printr-un singur operator , iar dicționarul grupează datele și se transformă direct în JSON.Comprehension-urile au făcut filtrările compacte , iar metoda .get a evitat o eroare la cheia risc , care lipsește la sarcinile cu cron.